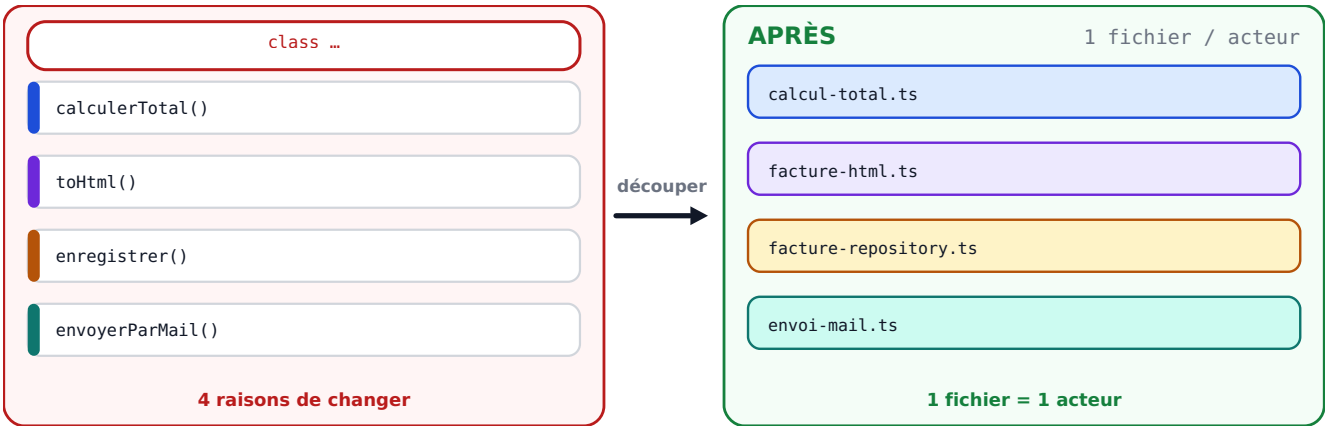
SOLID · LE S

Single /Responsibility

Une seule raison de changer

Tout le principe en schémas : un code couleur pour chaque type de responsabilité, 10 cas réels découpés sous tes yeux, et les pièges du sur-découpage.



1	La définition, en un seul schéma	2
2	Les deux tests : « qui demande ? » et « et »	3
3	Les 4 axes de découpe (métier · base · vue · orchestration)	4
4	Anatomie de l'avant · anatomie de l'après	5-6
5	10 cas réels, avant / après en images + code	7-11
6	Le reste de la boîte en un coup d'œil	12
7	Les pièges : sur-découper · mal nommer	13
8	Arbre de décision · récap visuel	14-15

> Pour réviser : les schémas seuls suffisent. Repère surtout le **code couleur** — bleu = métier, ambre = base, violet = affichage, vert d'eau = service externe, gris = orchestration. Il est identique sur les 15 pages.

« Une classe, une équipe, une raison de changer. »

Single Responsibility Principle

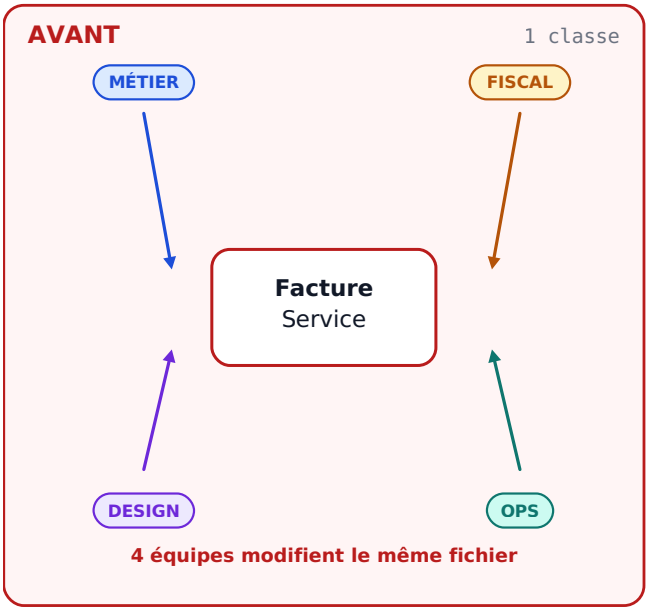
1 · LE PRINCIPE

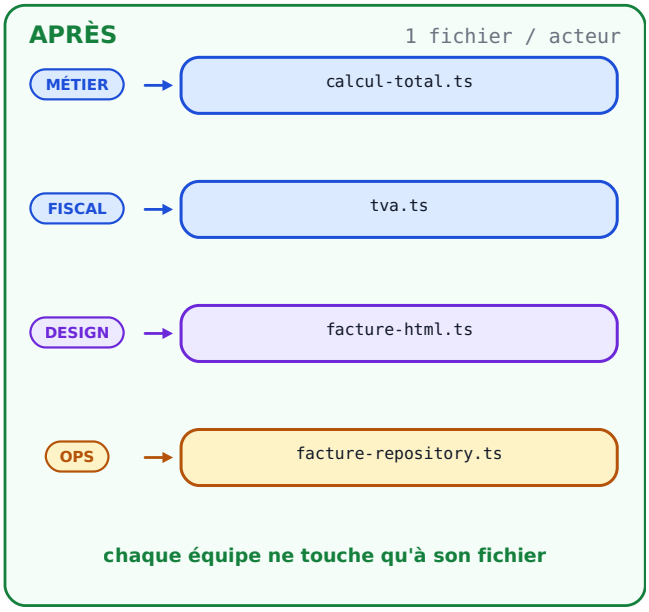
Une seule raison de changer

« Une classe ne doit avoir qu'une seule raison de changer » — c'est-à-dire **un seul acteur** qui peut exiger qu'on la modifie.

Une classe ne doit avoir qu'UNE seule raison de changer.

Une « raison de changer » = un acteur (une équipe, un métier) qui demande le changement.





1 acteur → 1 module → 1 raison de changer

1 · Ce que le SRP n'est pas

« Une classe = une méthode », « 20 lignes max ». Ce sont des **tailles**, pas des responsabilités.

2 · Ce que le SRP est

Un **acteur** unique : le service compta, l'équipe design, la DBA… Une seule source de demande de changement.

3 · Le bénéfice concret

Une demande de la compta ne peut plus **casser** l'affichage ni la base : elles ne vivent plus dans le même fichier.

AVANT — une classe, quatre métiers

```
class Facture {
  calculerTotal() { /* fiscal */ }
  toHtml()        { /* design */ }
  enregistrer()   { /* DBA */ }
  envoyerParMail(){ /* ops */ }
}
```

APRÈS — une classe, un acteur

```
// facture/calcul-total.ts     (fiscal)
// facture/facture-html.ts     (design)
// facture/repository.ts       (DBA)
// facture/envoi-mail.ts       (ops)

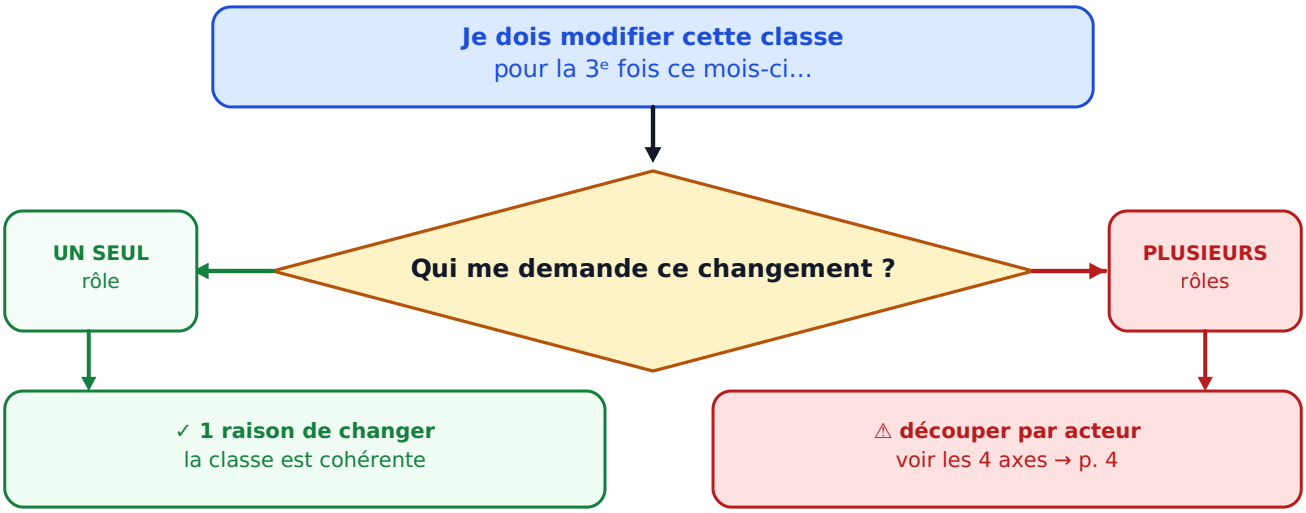
// et un seul fichier les appelle : encaisser.ts
```

DEUX RESPONSABILITÉS COHABITENT SI…	…ALORS QU'UNE SEULE DEVRAIT ÊTRE CONCERNÉE
la classe a besoin d'une base dans ses tests unitaires	une règle métier n'a besoin de **rien** pour se tester
un changement d'affichage casse un calcul	aucun test ne devrait relier ces deux mondes
deux équipes demandent des changements le même sprint	une seule équipe doit pouvoir l'exiger

2 · LES DEUX TESTS

« Qui demande ? » et « y a-t-il un et ? »

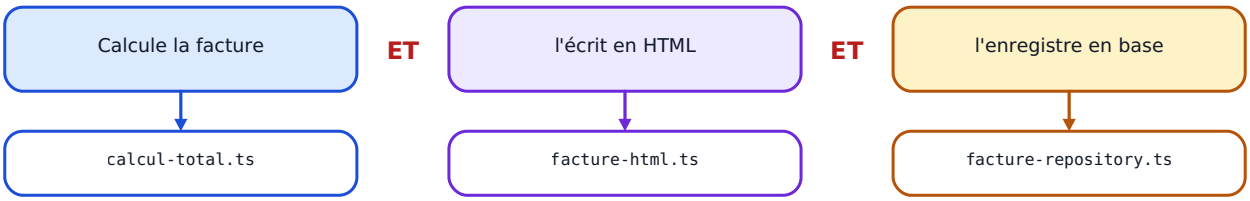
Deux questions suffisent pour savoir si une classe porte trop de choses.



Deuxième test : décris la classe en une seule phrase.

Si tu dois dire « et », ce sont déjà deux classes.

✗ une phrase = trois responsabilités



✓ trois fichiers, trois acteurs

A · Le test de l'acteur

Si deux personnes **différentes** peuvent demander un changement, il y a deux responsabilités.

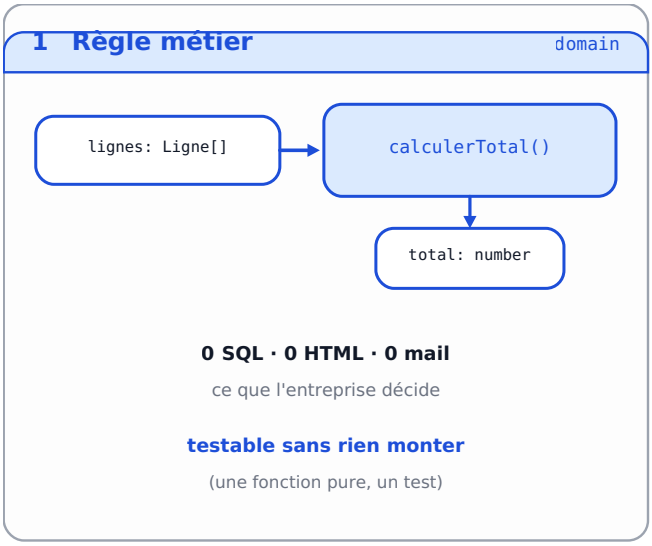
B · Le test de la phrase

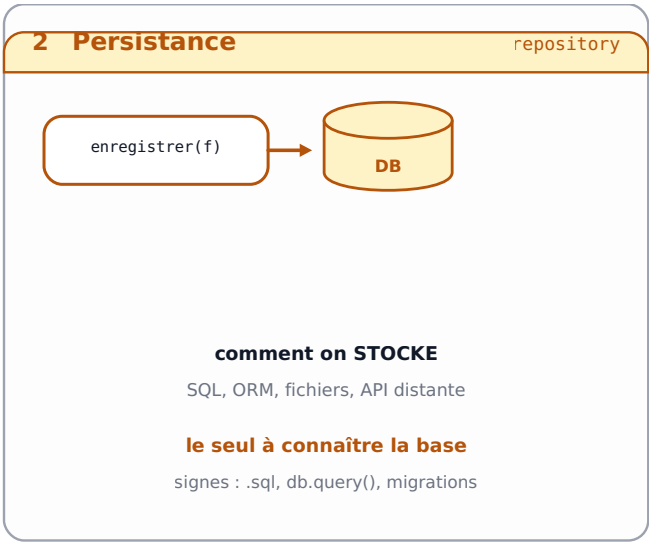
Une responsabilité se décrit en **une phrase sans « et »** : c'est le test le plus rapide.

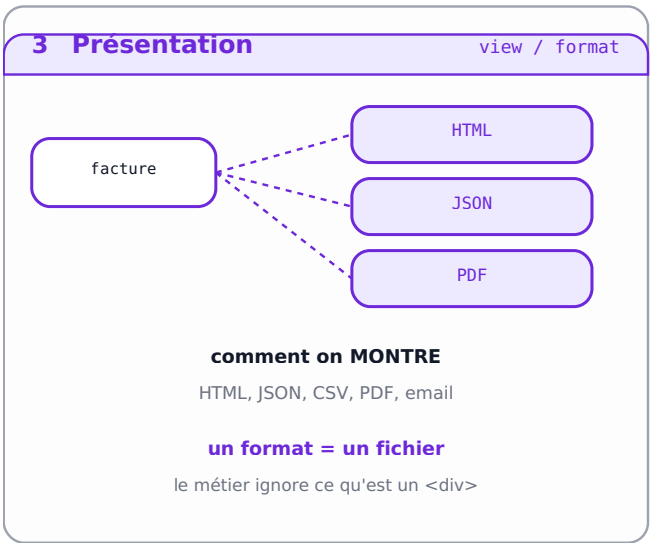
LE SYMPTÔME DANS TON CODE	LA RESPONSABILITÉ CACHÉE	AXE
`db.query()` dans une règle	persistance	2
il faut redéployer pour changer un taux	règle métier codée en dur	1
un changement d'affichage casse un calcul	présentation mélangée au métier	3
5 mocks dans le test d'une fonction	orchestration non isolée	4
11 auteurs sur le même fichier (git blame)	les quatre à la fois	1 → 4

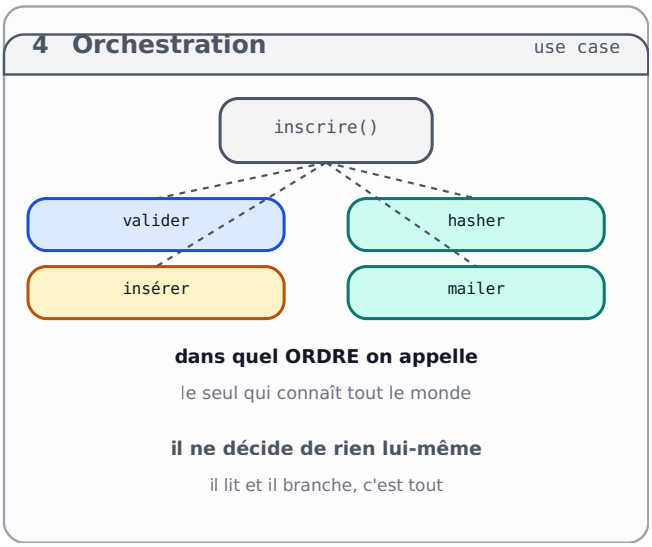
Les 4 axes de découpe (et rien d'autre)

Quand une classe est trop grosse, la question n'est pas « combien de lignes ? » mais « **de quelle nature** est ce code ? ».







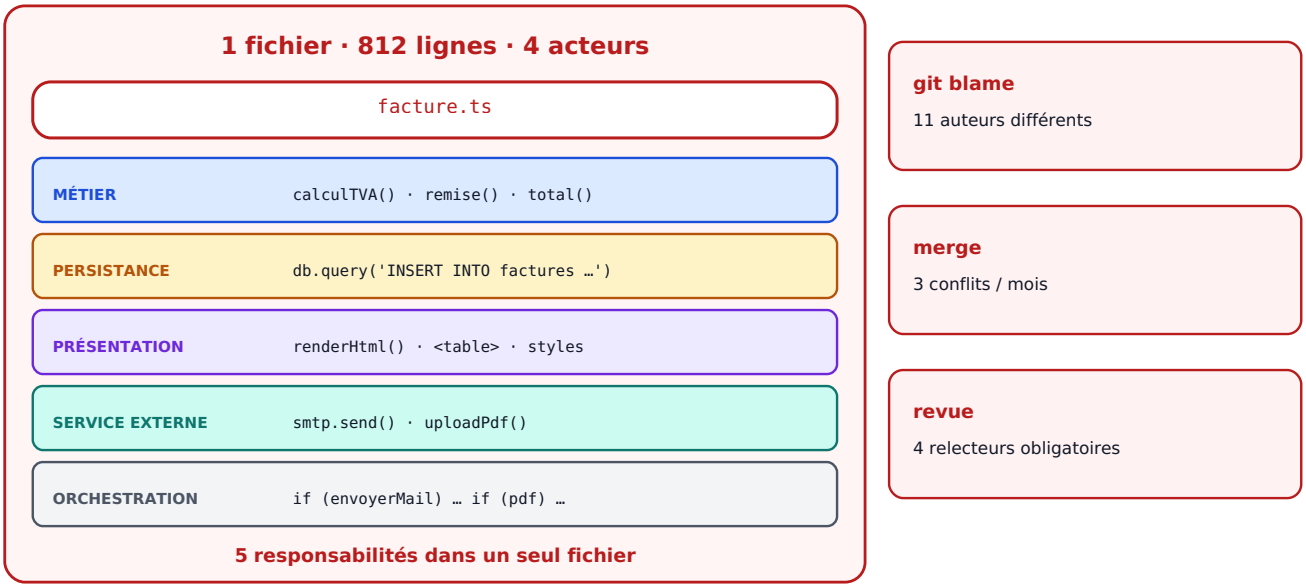


Une 5e famille existe : les **services externes** (mail, paiement, stockage, SMS). On les sépare pour la même raison qu'une base de données : ce sont des dépendances qui ont leur propre rythme de changement, leurs pannes et leurs tests à elles. On les met derrière une petite interface.
Et le signe qu'il faut découper : deux changements récents demandés par deux équipes différentes ont touché les mêmes 40 lignes.

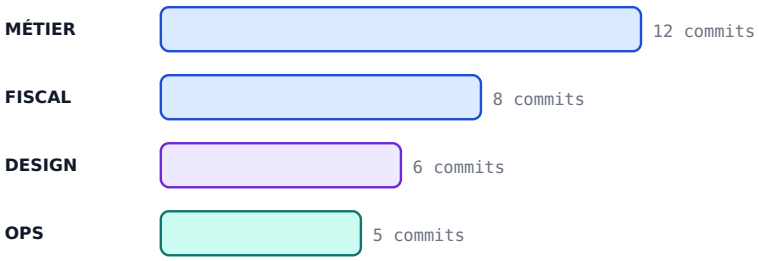
LE SYMPTÔME DANS TON CODE	L'AXE	CE QUE TU SORS
`db.query(…)` au milieu d'une règle	**2 · Persistance**	un `repository`
`<div>` ou `toFixed()` dans le métier	**3 · Présentation**	un rendu ou un `dto`
`smtp.send()` au milieu du calcul	**5 · Service externe**	un `gateway` derrière une interface
`if (envoyerMail) … if (pdf) …`	**4 · Orchestration**	un use case de 10 lignes

Anatomie de l'avant : le fichier que tout le monde touche

À quoi ressemble le problème — et pourquoi le sur-découpage n'est pas le sujet : le sujet, c'est **qui** demande.



Qui modifie ce fichier sur un mois



Tout le monde touche le même fichier :

chaque demande croise les 4 autres, et chaque revue doit tout comprendre.

→ 1 demande = 4 relecteurs

A · Le coût invisible

Chaque demande oblige à **comprendre quatre métiers** avant d'écrire une ligne.

B · Le coût de revue

Un diff de 40 lignes dans un fichier de 800 : quatre relecteurs, et en pratique aucun.

C · Le coût des tests

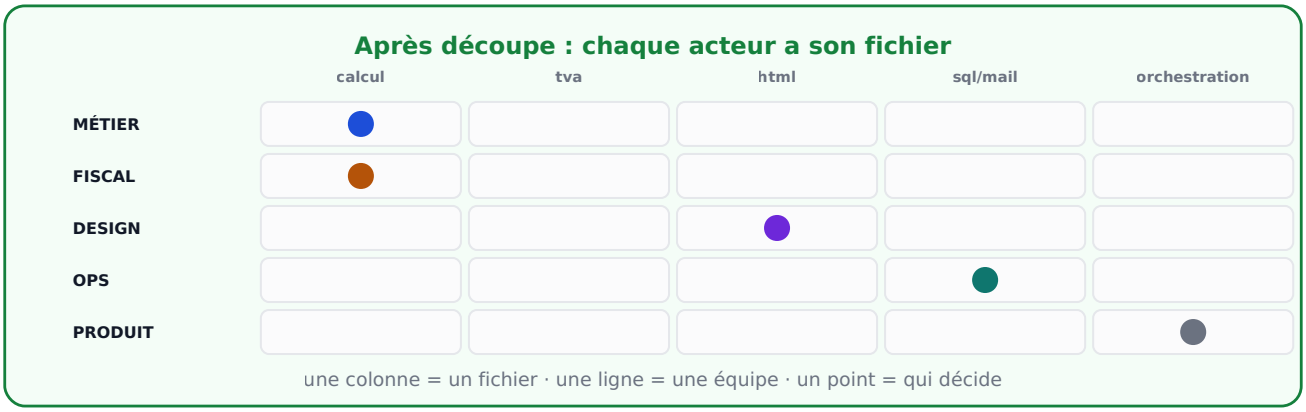
On ne peut plus tester la règle seule : il faut une base, un SMTP, un template.

UNE DEMANDE D'UN SPRINT TYPE	QUI LA PORTE	FICHIER TOUCHÉ
« la TVA passe à 5,5 % sur les livres »	compta	`facture.ts`
« on met le logo sur le PDF »	design	`facture.ts`
« on archive les factures de plus de 10 ans »	DBA	`facture.ts`
« on relance les impayés par mail »	ops	`facture.ts`

5 · LA CIBLE

Anatomie de l'après : une matrice sans collision

Après découpe, chaque équipe a son fichier — et les demandes ne se croisent plus.



Ce que la découpe change concrètement

	avant	après
Conflits de merge	3 / mois	**0**
Relecteurs par PR	4 équipes	**1 équipe**
Temps pour changer un taux de TVA	2 jours	**1 heure**
Risque de casser l'affichage	à chaque calcul	**aucun**

Le critère de réussite : quand la compta demande un changement de taux, tu n'ouvres **qu'un fichier** — et tu n'as ni à comprendre ni à retester l'affichage.

A · Un fichier, une équipe

La PR ne touche qu'un module : la revue tient en 10 minutes, sans expertise croisée.

B · Des tests indépendants

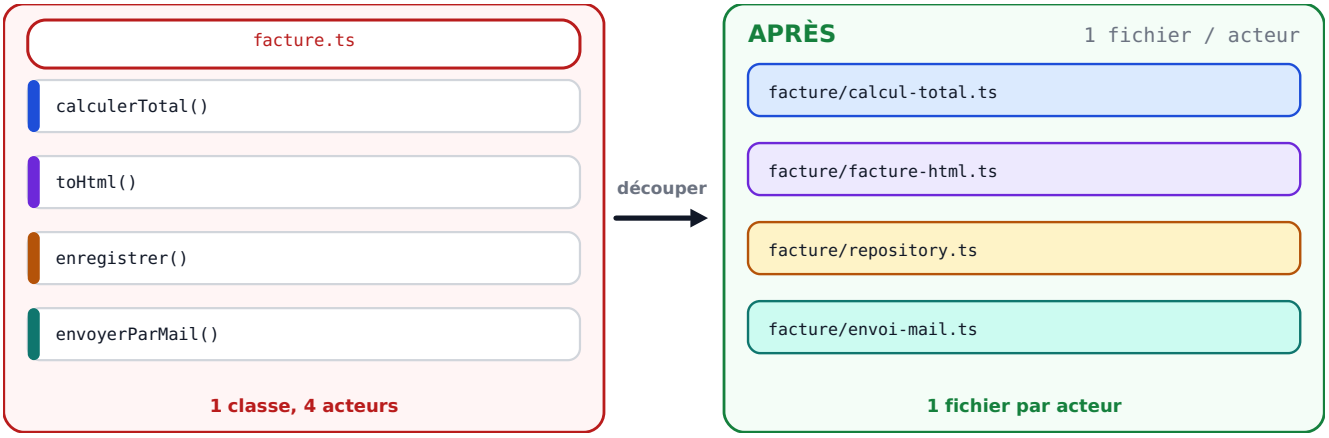
Le calcul se teste en 2 ms, la base en intégration, le mail avec un faux.

C · Un nouveau venu

Il lit **un dossier** pour comprendre la facturation — pas quatre couches techniques.

1 La facture (le classique)

calcul + HTML + SQL + mail dans une classe



AVANT

```
// facture.ts
class Facture {
  calculerTotal()  { /* TVA, remises */ }
  toHtml()         { /* <table>, styles */ }
  enregistrer()    { /* INSERT INTO factures */ }
  envoyerParMail() { /* SMTP, template */ }
}
```

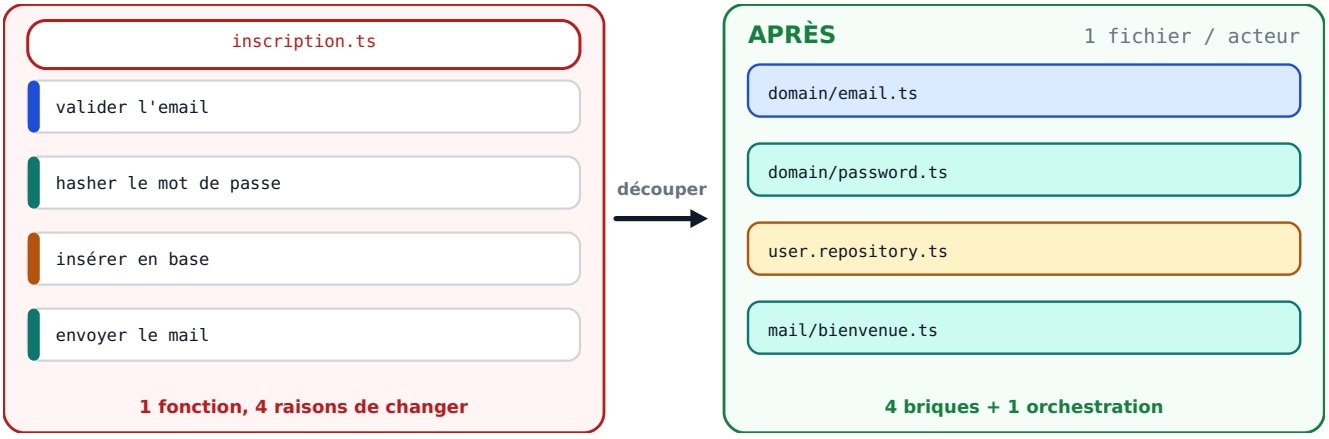
APRÈS

```
// facture/calcul-total.ts  (fiscal)
export function calculerTotal(l: Ligne[]) {
  return l.reduce((t, x) => t + x.prix * x.qte, 0);
}
// facture/facture-html.ts  (design)
export function rendreHtml(f: Facture) { … }
// facture/repository.ts    (DBA)
export function enregistrer(f: Facture) { … }
// facture/envoi-mail.ts    (ops)
export function envoyer(f: Facture) { … }
```

À retenir : la compta peut changer un taux **sans ouvrir** le fichier qui contient du HTML, et le design peut changer la mise en page sans risquer de casser un calcul.

2 Inscription utilisateur

règle + crypto + base + email dans une fonction



AVANT

```
// inscription.ts
async function inscrire(dto: Dto) {
  if (!dto.email.includes("@")) throw new Error("email");
  const hash = await bcrypt.hash(dto.password, 10);
  const user = await db.insert("users", { ...dto, hash });
  await sendgrid.send(user.email, "Bienvenue !");
  return user;
}
```

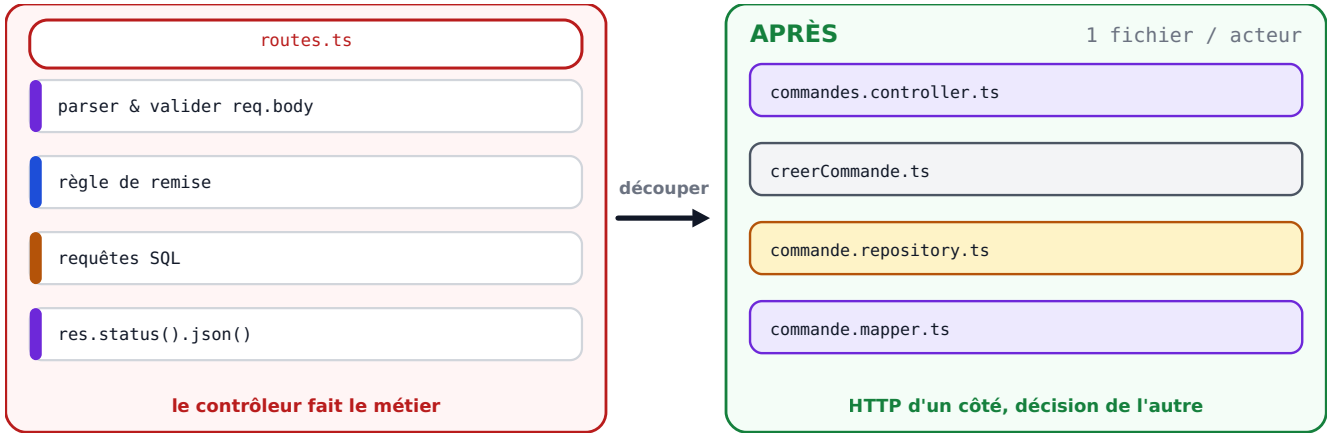
APRÈS

```
// domain/email.ts
export const emailValide = (e: string) => /@/.test(e);
// domain/password.ts
export const hasher = (p: string) => bcrypt.hash(p, 10);
// mail/bienvenue.ts  ·  user.repository.ts
export const inserer = (d: Dto) => db.insert("users", d);
// usecases/inscrire.ts  ← orchestration
export async function inscrire(dto: Dto) {
  emailValide(dto.email);
  const hash = await hasher(dto.password);
  const user = await inserer({ ...dto, hash });
  await bienvenue(user);
}
```

À retenir : la règle de validation d'email est utilisée **à la connexion et au reset** aussi. Tant qu'elle vit dans `inscrire()`, elle est dupliquée ou inaccessible.

3 Contrôleur HTTP obèse

toute la logique dans le handler de route



AVANT

```
// routes.ts — le contrôleur obèse
router.post("/commandes", async (req, res) => {
  if (!req.body.clientId) return res.status(400).end();
  const client = await db.client(req.body.clientId);
  const total = req.body.lignes.reduce(somme, 0);
  const net = client.vip ? total * 0.9 : total;
  await db.insert("commandes", { total: net });
  res.status(201).json({ total: net });
});
```

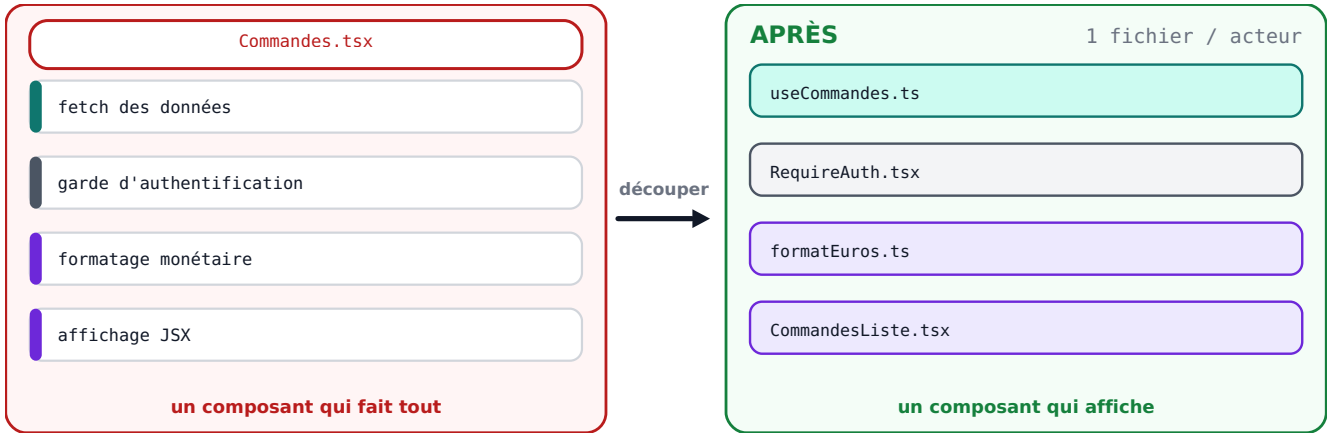
APRÈS

```
// commandes.controller.ts  (HTTP seul)
router.post("/commandes", async (req, res) => {
  const cmd = parseCommande(req.body);     // validation
  const { id, total } = await creerCommande(cmd);  // métier
  res.status(201).json({ id, total });
});
// creerCommande.ts
export async function creerCommande(c: Cmd) {
  const client = await clients.trouver(c.clientId);
  const net = appliquerRemise(client, totaliser(c));
  return commandes.enregistrer({ ...c, total: net });
}
```

À retenir : la règle de remise devient **appelable** depuis un job, un import, un test — pas seulement depuis une requête HTTP. C'est le signe qu'elle a enfin sa place.

4 Composant React

fetch + auth + format + affichage dans le même fichier



AVANT

```
// Commandes.tsx
function Commandes() {
  const [cmd, setCmd] = useState([]);
  useEffect(() => { fetch("/api/cmd").then(r => r.json())
    .then(setCmd); }, []);
  if (!localStorage.token) return <Login />;
  return <ul>{cmd.map(c => <li>
    {c.total.toFixed(2).replace(".", ",")} €</li>)}</ul>;
}
```

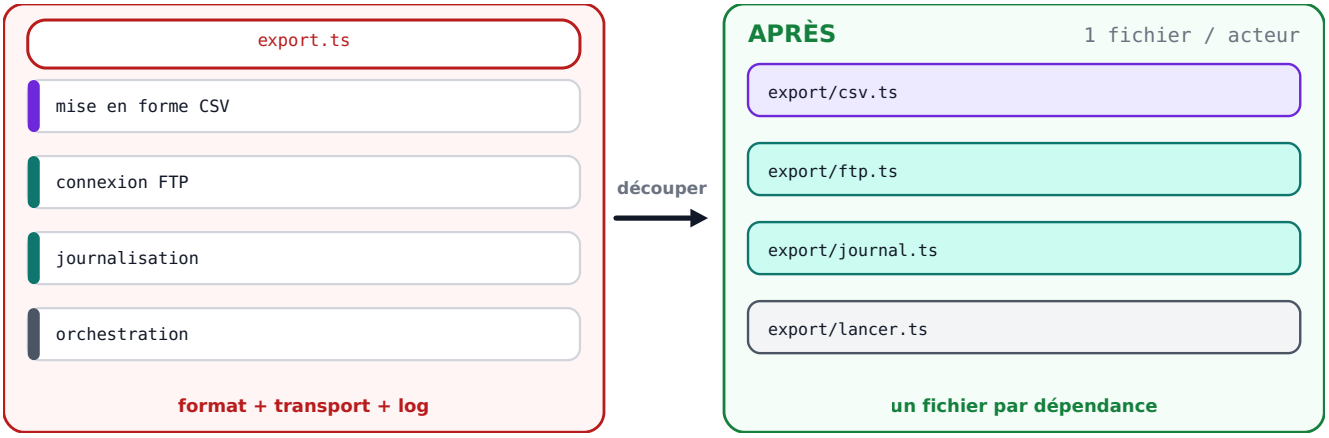
APRÈS

```
// useCommandes.ts   (données)
export const useCommandes = () => useQuery("/api/cmd");
// formatEuros.ts    (format)
export const formatEuros = (n: number) =>
  n.toFixed(2).replace(".", ",") + " €";
// CommandesListe.tsx (affichage seul)
function CommandesListe() {
  const { data } = useCommandes();
  return <ul>{data.map(c => <li key={c.id}>
    {formatEuros(c.total)}</li>)}</ul>;
}
```

À retenir : `formatEuros` sert maintenant dans 12 écrans, et la garde d'auth est une fois dans le routeur — plus dans chaque page.

5 Export CSV + dépôt FTP

format, transport et log enchaînés



AVANT

```
// export.ts
async function exportEtDepot(lignes: Ligne[]) {
  const csv = lignes.map(l => l.join(";")).join("\n");
  const conn = await ftp.connect(HOST, USER, PASS);
  await conn.put(Buffer.from(csv), "export.csv");
  conn.end();
  console.log(`export ok : ${lignes.length} lignes`);
}
```

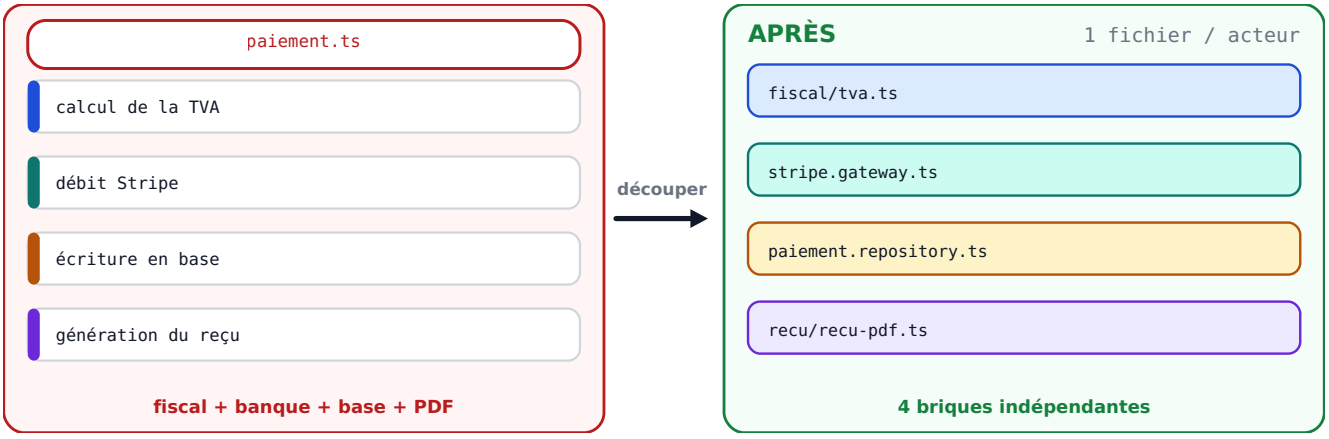
APRÈS

```
// export/lancer.ts ← orchestration
export async function lancer(l: Ligne[]) {
  await deposer("export.csv", versCsv(l));
  journaliser(`export ok : ${l.length} lignes`);
}
// csv.ts     versCsv()      testable sans réseau
// ftp.ts     deposer()      remplaçable par S3
// journal.ts journaliser()  un seul point de log
```

À retenir : le jour où l'on passe de FTP à S3, on change **un fichier de 20 lignes** — et le test du format CSV continue de tourner sans réseau.

6 Encaissement

TVA + prestataire + base + reçu dans une fonction



AVANT

```
// paiement.ts
async function payer(panier: Panier, cb: Carte) {
  const tva = panier.total * 0.2;
  const charge = await stripe.charge(cb, total + tva);
  await db.insert("paiements", { id: charge.id, tva });
  await pdf.gen(`Reçu ${charge.id}`);
}
```

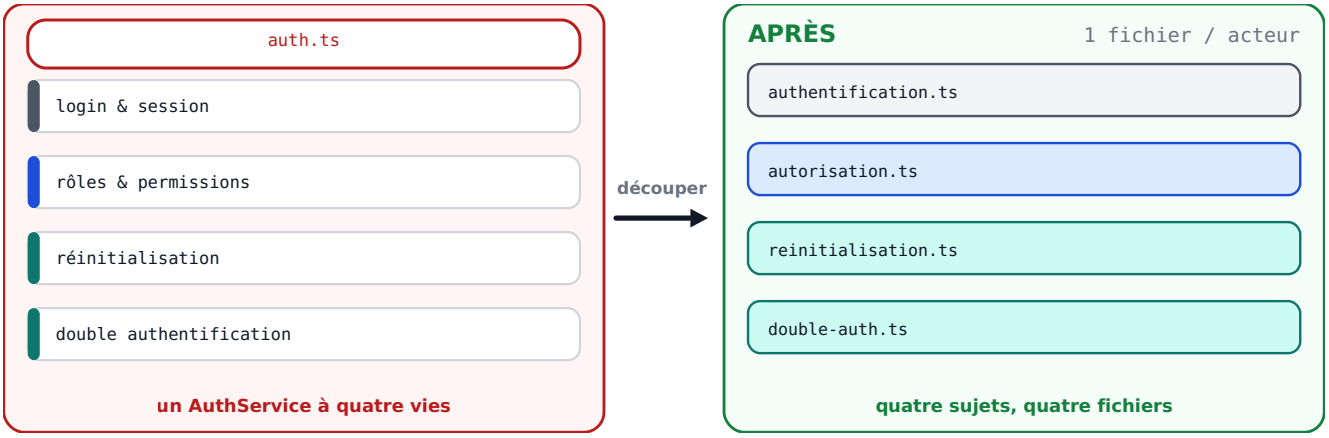
APRÈS

```
// usecases/encaisser.ts ← orchestration
export async function encaisser(p: Panier, cb: Carte) {
  const ttc = p.total + calculerTva(p.total);
  const charge = await debiter(cb, ttc);
  const paiement = await enregistrer({ charge, ttc });
  return rendreRecu(paiement);
}
// tva.ts · stripe.gateway.ts · repository.ts
// recu-pdf.ts   → 4 fichiers, 4 raisons
```

À retenir : le taux de TVA change **tous les ans**, Stripe change tous les 3 ans, la base jamais. Trois rythmes différents = trois fichiers différents.

7 Service d'authentification

login + permissions + reset + 2FA



AVANT

```
// auth.ts — 1 classe, 4 sujets
class AuthService {
  login(email, mdp)       { /* hash, JWT, session */ }
  peutVoir(user, res)     { /* rôles, ACL */ }
  reset(email)            { /* token, mail, TTL */ }
  verifier2fa(user, code) { /* TOTP */ }
}
```

APRÈS

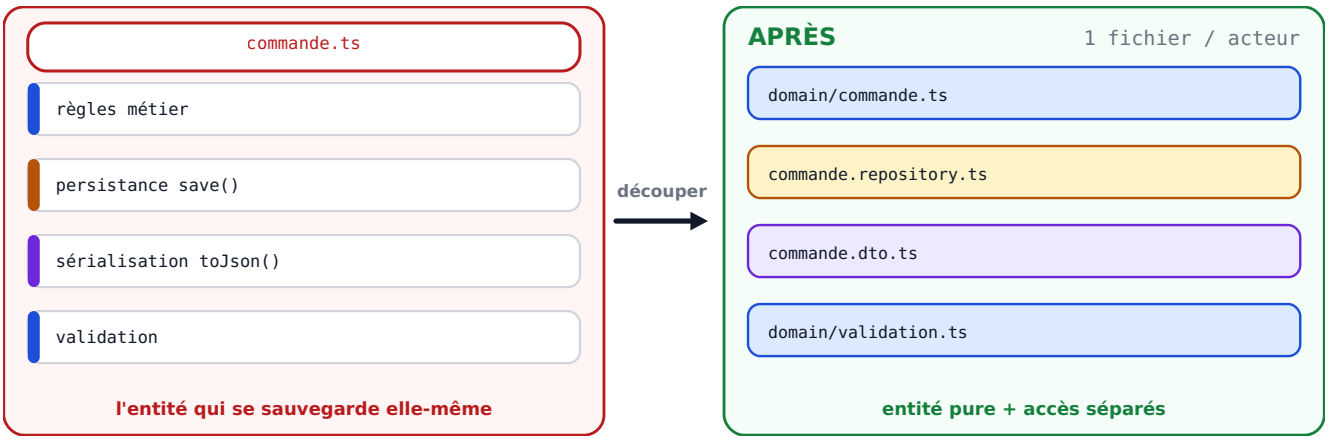
```
// authentification.ts  login / logout / session
// autorisation.ts      peutVoir(user, ressource)
// reinitialisation.ts  demander / confirmer
// double-auth.ts       activer / vérifier

// 4 fichiers, 4 propriétaires :
// sécurité · produit · support · sécurité
```

À retenir : chaque sujet a son **propre vocabulaire** (session, ACL, TTL, TOTP). Quatre vocabulaires dans une classe = quatre équipes qui se marchent dessus.

8 Entité qui se sauvegarde

le piège de l'Active Record



AVANT

```
// commande.ts — entité + ORM + API
class Commande {
  ajouterLigne(l) { this.lignes.push(l); }
  total()         { /* remises, frais de port */ }
  save()          { db.query("UPDATE commandes …") }
  toJson()        { return { …this, total: this.total() }; }
}
```

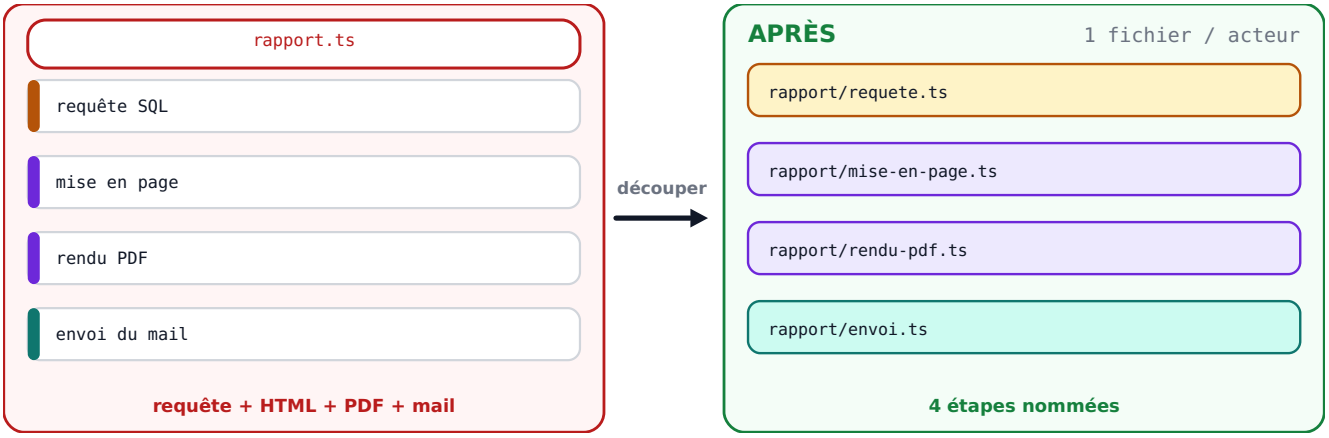
APRÈS

```
// domain/commande.ts — aucune dépendance
export const ajouterLigne = (c: Cmd, l: Ligne) => …
export const total = (c: Cmd) => remise(c) + port(c);
// commande.repository.ts
export const save = (c: Cmd) => db.update("commandes", c);
// commande.dto.ts
export const toJson = (c: Cmd) =>
  ({ id: c.id, total: total(c) });
```

À retenir : une entité qui appelle la base **ne se teste plus** sans base. Sortir `save()` et `toJson()` rend les règles testables en 2 ms.

9 Génération de rapport

SQL, template et envoi enchaînés



AVANT

```
// rapport.ts
async function genererRapport(mois: string) {
  const rows = await db.query(SQL_VENTES, [mois]);
  const html = "<h1>Rapport</h1>"
    + rows.map(ligne).join("");
  await pdf.render(html, "rapport.pdf");
  await mail.send("compta@…", "rapport.pdf");
}
```

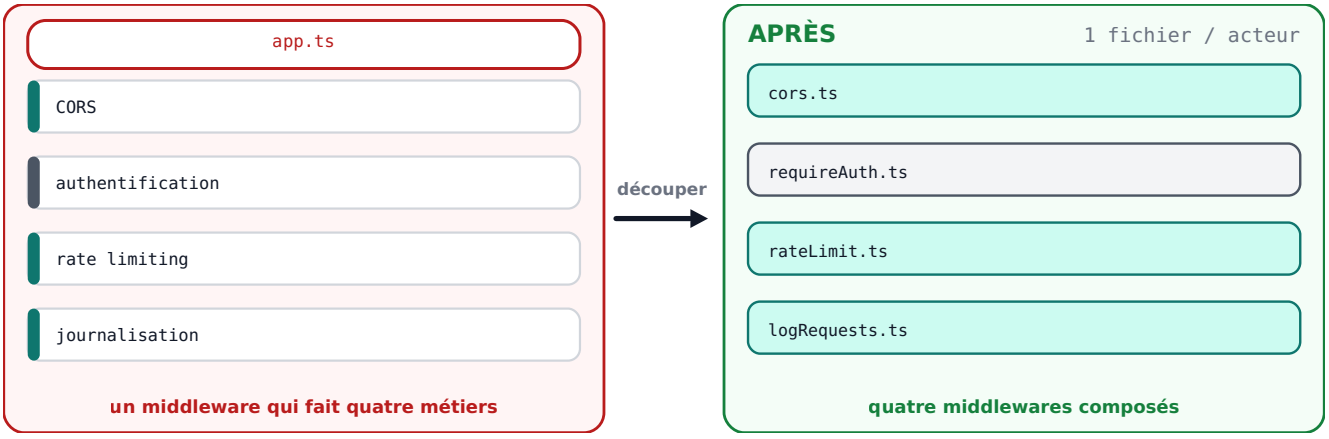
APRÈS

```
// rapport/generer.ts ← orchestration
export async function generer(mois: string) {
  const ventes = await chargerVentes(mois);      // SQL
  const html = rendreHtml(ventes);               // page
  await envoyerPdf("rapport.pdf", await rendrePdf(html));
}
// requete.ts · mise-en-page.ts · rendu-pdf.ts · envoi.ts
```

À retenir : la compta demande une **colonne de plus** → tu ouvres `mise-en-page.ts`. Le marketing demande un **envoi Slack** → `envoi.ts`. Jamais les deux à la fois.

10 Middleware fourre-tout

CORS + auth + quota + log dans un seul use



AVANT

```
// app.ts
app.use((req, res, next) => {
  res.setHeader("Access-Control-Allow-Origin", "*");
  if (!req.headers.authorization) res.status(401).end();
  if (compteur[req.ip]++ > 100) res.status(429).end();
  console.log(`${req.method} ${req.url}`);
  next();
});
```

APRÈS

```
// app.ts — 1 middleware = 1 responsabilité
app.use(cors());
app.use(requireAuth());
app.use(rateLimit(100));
app.use(logRequests());

// chacun se teste seul, se réordonne, se retire
// et se réutilise sur un autre service
```

À retenir : sous cette forme, **l'ordre est visible** et modifiable. Dans la version monolithique, changer l'ordre demandait de déplacer des lignes `if` — donc de risquer une régression.

6 · TOUR D'HORIZON

Le reste de la boîte, en un coup d'œil

Six fichiers que tu as forcément croisés : à gauche le symptôme, à droite la découpe.

DateHelper fourre-tout
format + calcul + fuseaux
	`format(d)`
	`ajouterJours(d, n)`
	`versParis(d)`
		→ `format.ts`
		`calculs.ts`
		`fuseaux.ts`
			3 usages distincts
			le backend ne gère pas les fuseaux

EmailSender obèse
envoi + template + retry
	`render(tpl, data)`
	`send(smtp, msg)`
	`retry(3, send)`
		→ `template.ts`
		`smtp.ts`
		`avecRetry.ts`
			3 dépendances
			le template change sans le SMTP

Validator qui normalise
vérifie ET transforme
	`estEmail(s)`
	`trimLower(s)`
	`sanitize(html)`
		→ `valider.ts`
		`normaliser.ts`
		`nettoyer.ts`
			2 responsabilités
			valider n'est pas corriger

utils.ts, le fichier god
tout le monde y met tout
	`formatDate()`
	`calculerTVA()`
	`slugify()`
		→ `date.ts`
		`fiscal.ts`
		`texte.ts`
			imports en pagaille
			1 changement = rebuild global

NotificationService
envoi + préférences + template
	`choisirCanal(u)`
	`render(msg)`
	`envoyer(canal)`
		→ `preferences.ts`
		`template.ts`
		`canaux.ts`
			3 acteurs
			produit / design / ops

routes.ts qui décide
routage + règles + SQL
	`router.get(…)`
	`if (user.admin)`
	`db.query(…)`
		→ `routes.ts`
		`policy.ts`
		`repository.ts`
			routage ≠ décision
			la règle sert aussi hors HTTP

A · Le réflexe de nommage

Un fichier = un nom de métier. Si tu n'arrives pas à le nommer, deux choses y vivent.

B · Le fichier de 400 lignes

Pas un problème en soi s'il n'a qu'un acteur : le SRP n'est pas une limite de taille.

C · En cas de doute

Ne découpe pas tant que le **2e acteur** n'a pas réellement frappé à la porte.

La question qui marche à tous les coups : « si la compta, la DBA et le designer devaient chacun demander un changement cette semaine, est-ce qu'ils toucheraient le même fichier ? »

7 · LES PIÈGES

Trop fusionner… et trop découper

Le SRP se joue sur un curseur : trop peu de fichiers, ou trop. Le bon point n'est pas une taille, c'est un **acteur**.

TROP FUSIONNÉ

1 fichier, 4 acteurs

- métier
- base
- html
- mail

1 changement = 800 lignes à lire

JUSTE

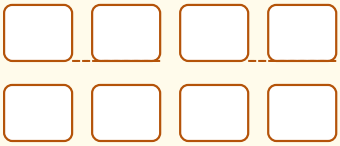
1 acteur = 1 module



1 changement = 1 fichier

TROP DÉCOUPÉ

30 classes de 5 lignes



1 changement = 8 fichiers

Le second piège : découper par couche technique au lieu d'acteur

✗ par couche technique

```
src/
models/ ← métier + SQL mélangés
services/ ← tout ce qui reste
utils/ ← le grenier fourre-tout
```

un changement métier touche 3 dossiers

✓ par acteur / capacité

```
src/
facturation/ calcul + html + repo + mail
expedition/ calcul + html + repo
auth/ login + reset + 2fa
```

un changement métier touche un dossier

1 · Le SRP n'est pas une taille

« 1 classe = 1 méthode », « 50 lignes max » : ce sont des **seuils arbitraires**. Un module de 300 lignes peut avoir une seule raison de changer.

2 · Le vrai test

Deux personnes **différentes** peuvent-elles demander un changement ? Si oui : deux responsabilités.

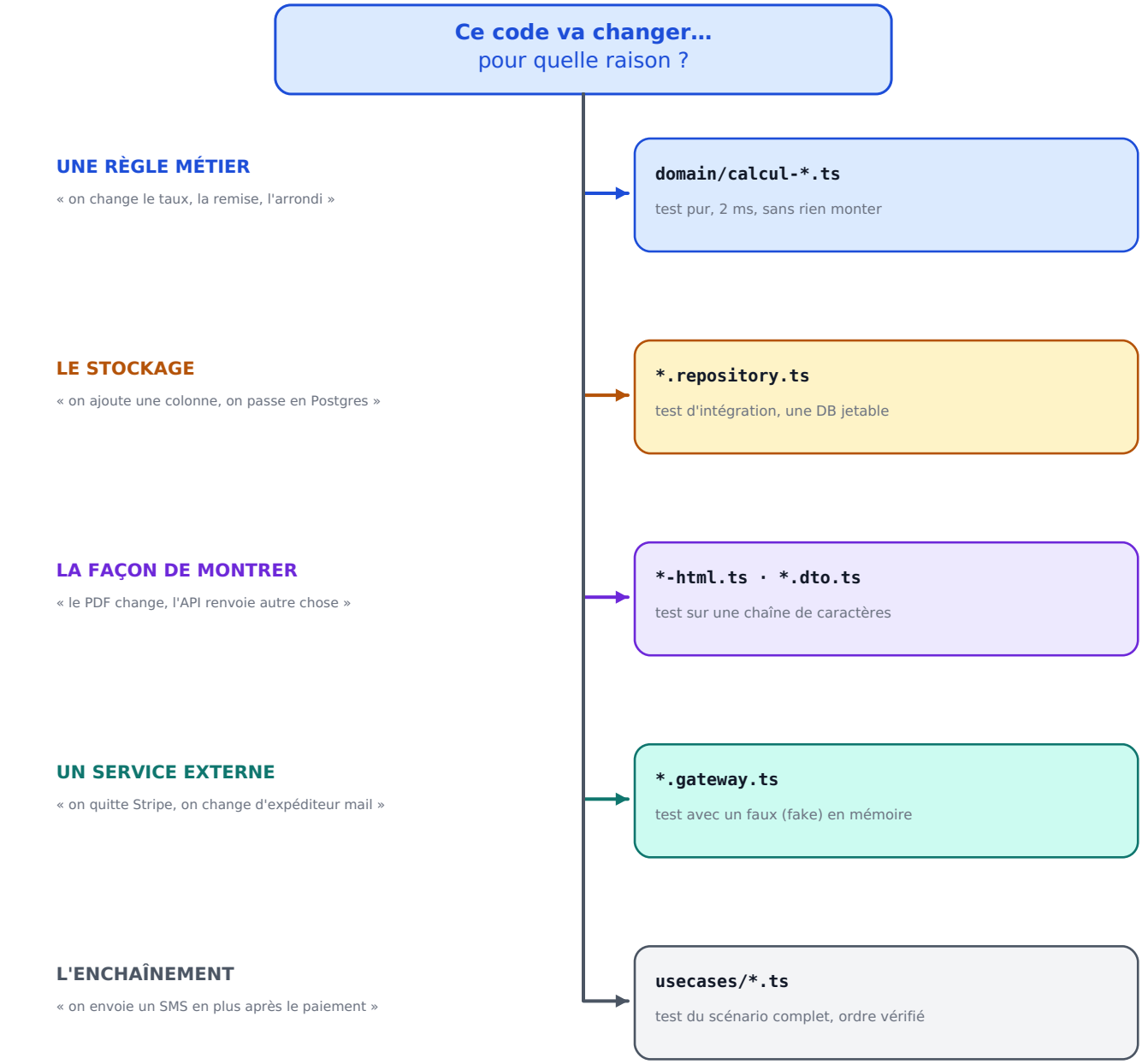
3 · Le coût du sur-découpage

Chaque classe ajoute un saut à lire. Un changement qui traverse 8 fichiers coûte plus cher que 80 lignes relues.

CE QUE TU OBSERVES	CE QUE ÇA DIT VRAIMENT	LE GESTE
Le fichier a 11 auteurs (`git blame`)	quatre acteurs sur un seul fichier	découper par acteur
Chaque test mocke 5 objets	l'orchestration est mêlée au métier	extraire un use case + des interfaces
Il faut redéployer pour changer un taux	une valeur métier est dans le code	la sortir en configuration (pont vers l'OCP)
Un changement touche 8 fichiers	sur-découpage	refusionner par cas d'usage

Quel fichier pour ce code ?

La question n'est pas « où ranger ? » mais « **qui peut demander ce changement** ? ».



Une seule raison → un seul fichier. Plusieurs raisons → plusieurs fichiers.

Et si le code ne change jamais pour personne : on ne le découpe pas.

A · Un fichier = un acteur

Pas un fichier par fonction ni par classe : un fichier par **raison de changer**.

B · L'orchestrateur ne décide pas

S'il contient un `if` métier, la règle est au mauvais endroit.

C · Les noms disent le métier

`facturation/` plutôt que `helpers/` : le nom est le premier test.

9 · RÉVISER

Le code couleur à retenir

Cinq couleurs, cinq raisons de changer. Si tu retiens ça, tu retiens le SRP.



En une phrase ?
Une classe, une raison de changer — c'est-à-dire un seul acteur qui peut l'exiger.

Les deux tests ?
« Qui demande ce changement ? » et « y a-t-il un et dans ma phrase ? »

Les 4 axes ?
Métier · persistance · présentation · orchestration (+ services externes).

Le symptôme ?
Le même fichier modifié par 4 équipes, ou 3 conflits de merge par mois dessus.

Le contre-poids ?
Trop découper coûte aussi : pas de seam sans acteur réel derrière.

La preuve que c'est bon ?
La compta change un taux, un seul fichier s'ouvre, l'affichage ne bouge pas.

CHECKLIST AVANT DE VALIDER UNE DÉCOUPE	SI NON →
Chaque module a-t-il **un seul acteur** identifié ?	c'est encore une responsabilité partagée
Le nom du module dit-il **le métier**, pas la technique ?	tu as découpé par couche, pas par acteur
Le module métier est-il **testable sans base ni réseau** ?	une dépendance est remontée trop haut
L'orchestration ne contient-elle **aucune règle** ?	elle va changer pour deux raisons

Et si tu oublies tout le reste : avant d'écrire une fonction, demande-toi **qui** te demandera de la changer. S'il y a deux réponses, tu écris deux fichiers.